TIZADA PRO

Crear la plantilla en Illustrator o CorelDRAW desde su sistema

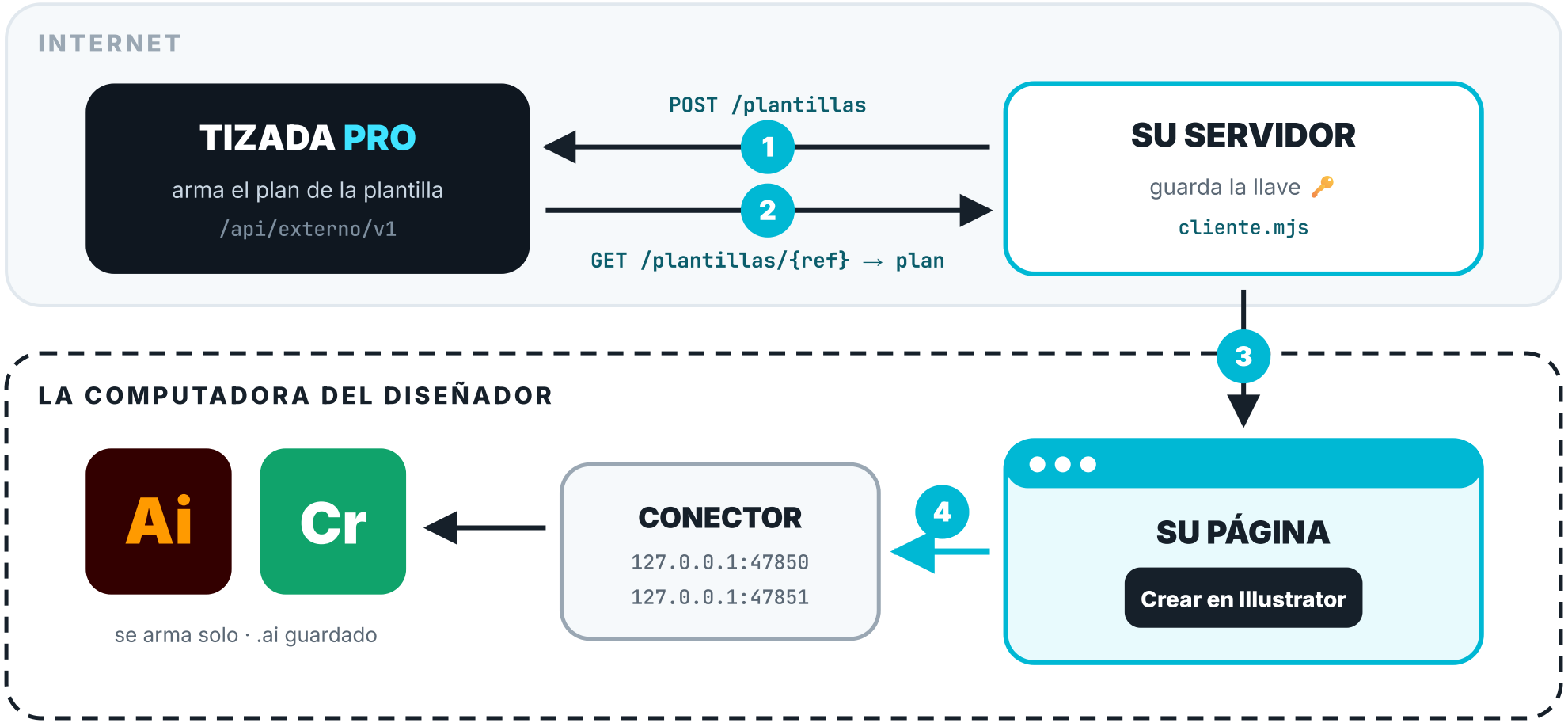
Lo mismo que el botón «Crear en Illustrator / CorelDRAW» de TIZADA, ordenado por ustedes.

3	**2**	**0**
pasos	rutas en su servidor + 1 botón en su página	cambios en lo que ya mandan

Detalle: API.md §6.b · código: codigo/crear_en_programa.js y codigo/cliente.mjs

Cómo se une

TIZADA dice **cómo** se crea. Su sistema sólo lo lleva hasta el programa.



1 su servidor pide la plantilla

2 TIZADA la arma y devuelve el plan

3 su página lo pide a SU servidor

4 su página se lo da al conector → el programa

La llave nunca sale de su servidor · el conector sólo escucha en esa computadora

0 Instalar el conector, una vez

En cada computadora donde se diseña. Doble clic → «Instalar».



Windows · .exe

Illustrator

`GET /conectores/illustrator`



Mac · .zip

Illustrator (Mac)

`GET /conectores/illustrator-mac`



Windows · 2022+

CorelDRAW

`GET /conectores/corel`

? ¿Ya está? Desde esa computadora: `GET http://127.0.0.1:47850/estado` (Illustrator) · `:47851/estado` (CorelDRAW)

↻ Si su `version` no es la que dice TIZADA → volver a instalar. **El código ya lo avisa.**

La lista con versión y nombre de archivo: GET /api/externo/v1/conectores

1 Su servidor: pedir y servir

Con la llave. Usa `codigo/cliente.mjs` (Node 18+, sin dependencias).

1 Pedir la plantilla
los diseños y sus variables, como en el pedido

2 Esperar «listo»
el robot de TIZADA la arma en segundos

A Ruta para su página: el estado
`GET /plantillas/:ref` (en su servidor)

B Ruta para su página: cada archivo
`GET /plantillas/:ref/archivos/:nombre` (en su servidor)

`archivos: illustrator · corel · guia`

`conectores: a dónde · versión`

```
import { pedirPlantilla, esperarPlantilla, archivoPlantilla }
  from './cliente.mjs'

// 1 · pedir
await pedirPlantilla({
  referencia: 'PL-0001',
  disenos: [{ nombre: 'JUGADOR', variables: ['v_redondo'] }],
  talles: { modo: 'todos' },
  escala: 100
})

// 2 · esperar → {estado:'listo', archivos, conectores}
const est = await esperarPlantilla('PL-0001')

// A · a su página: el estado tal cual
app.get('/plantillas/:ref', … res.json(est))

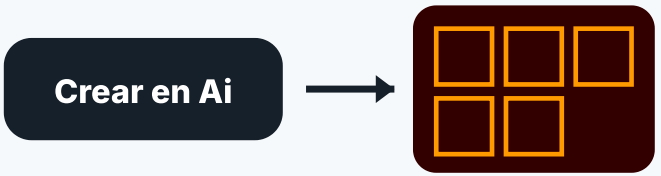
// B · a su página: el archivo (comprobado con sha256)
app.get('/plantillas/:ref/archivos/:nombre', …
  res.send(await archivoPlantilla(archivo)))
```

2 Su página: un botón

En la computadora del diseñador. Usa `codigo/crear_en_programa.js` (navegador, sin dependencias).

```
import { crearEnPrograma } from './crear_en_programa.js'

boton.onclick = async () => {
  const estado = await (await fetch('/plantillas/PL-0001')).json()
  const r = await crearEnPrograma({
    programa: 'illustrator',   // o 'corel'
    estado,
    diseno: 'JUGADOR',
    traer: (a) => fetch('/plantillas/PL-0001/archivos/'
                  + encodeURIComponent(a.nombre)).then(x => x.text()),
    alAvanzar: (texto) => cartel(texto)
  })
  // r.mesas · r.archivos · r.avisos
}
```



una mesa por pieza, con capas y contornos

Devuelve

`ok: true` `mesas: 15`

`archivos: ['JUGADOR.ai']`

Si algo falla, `error.codigo`:

`no-lista` `sin-archivos`

`sin-conector → instalador` `programa`

✓ Lo que hay que saber

Seis reglas. Con esto alcanza.

- ✓ **Illustrator abierto.** CorelDRAW (2022+) se abre solo.
- ✓ **Permitir la red local** la primera vez que el navegador lo pregunte.
- ✓ **El plan va tal cual.** No se toca ni se rearma: TIZADA ya dice cómo es.
- ✓ **Sin conector:** la `guia` .ai se abre en cualquier Illustrator.
- ✗ **La llave nunca en el navegador.** Sólo en su servidor.
- ✗ **Sin cabeceras propias** hacia el conector: sólo `Content-Type`.

Probado	Nada se rompe	Detalle
de punta a punta, con un conector de prueba	lo que ya mandan sigue igual	`API.md §6.b` · `CAMBIOS.md` 27/10